



2. Consider the following SDT. Draw the Annotated Parse Tree and Evaluate the values of the attributes type and width along with variable offset next to the appropriate non-terminal nodes for the following String:

int x; record {int a; float b;}; float [5] [7] z;

Afterwards draw the memory diagram along with actual location and data

Note: Offsets are given in decimal. Memory location is also given in decimal. Every memory unit is of 1 byte. The width of int and float is also given in byte in the SDT. The variable memory\_addr refers to the starting position of the memory address where the variables will be stored.

SDT:

$P \rightarrow \{ \text{offset} = 0; \text{memory\_addr} = 58342 \} D$

$D \rightarrow T \text{ id } \{ \text{top.push(id.lexeme, T.type, offset); } D_1; \text{offset} = \text{offset} + T.\text{width}; \}$

$D \rightarrow \epsilon$

$T \rightarrow \text{record } \{ \{ \text{Elex.push(top); top = new Elex(); } D' \} \{ T.type = \text{record(top); } T.\text{width} = \text{offset} - \text{Stack.push(offset); offset = 0; } \}$

$T \rightarrow B [ \{ t = B.type; w = B.width \} ] C$   $(T.type = C.type; T.width = C.width)$

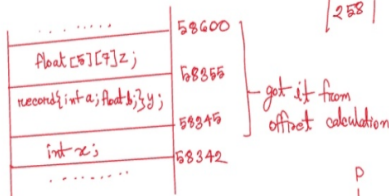
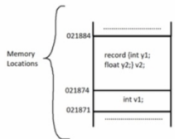
$B \rightarrow \text{int}$   $\{ B.type = \text{integer; } B.width = 3; \}$

$B \rightarrow \text{float}$   $\{ B.type = \text{float; } B.width = 7; \}$

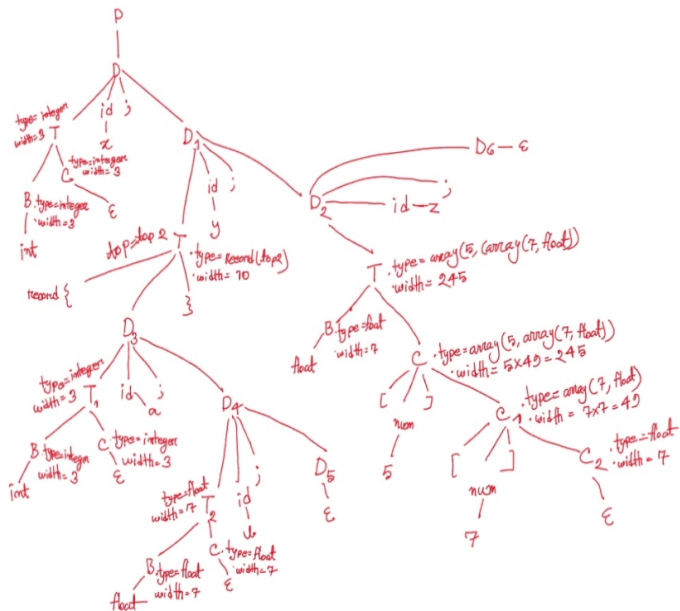
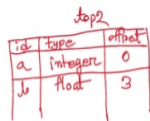
$C \rightarrow \epsilon$   $\{ C.type = \epsilon; C.width = 0; \}$

$C \rightarrow [ \text{mem} ] C_1$   $\{ C.type = \text{array(mem.value, C_1.type); } C.width = \text{mem.value} \times C_1.width; \}$

Example of Memory Diagram for String: int v1; record {int y1; float y2;}; v2; where memory\_addr = 021871; in the SDT.



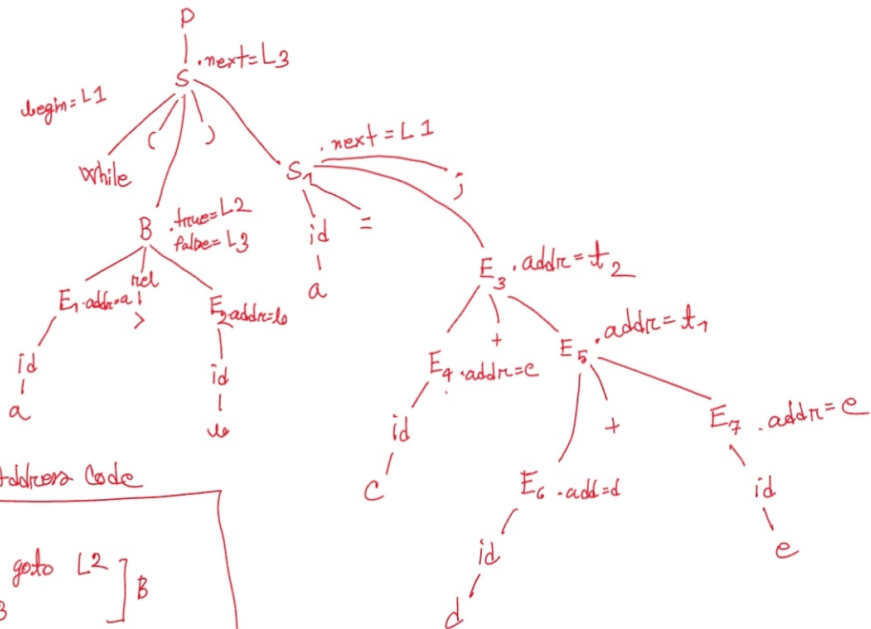
memory address



3. Consider the following SDD. Draw the Annotated Parse Tree and Evaluate it for the String: `while (a > b) a = c + d + e;`

Note: Here new Temp() creates an instance of compiler generated temporary variables which is 't' with a subscript 1, 2, 3 and so on. For example, the first time an instance is created it will create  $t_1$ , next time it is called it will create  $t_2$  and so on. Furthermore, the newLabel() method creates a label which is 'L:' with a subscript that follows similar logic as the temporary variable, for example,  $L_1$ ,  $L_2$ , and so on. If the newLabel() method is assigned to a variable like `begin = newLabel()`, it will name the label as the assigned variable like "begin:"

| PRODUCTION                      | SEMANTIC RULES                                                                                                                                                                                                         |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $P \rightarrow S$               | $S.next = newLabel()$<br>$P.code = S.code \parallel label(S.next)$                                                                                                                                                     |
| $S \rightarrow id = E ;$        | $S.code = E.code \parallel$<br>$gen(top.get(id.lexeme) '=' E.addr)$                                                                                                                                                    |
| $S \rightarrow while ( B ) S_1$ | $begin = newLabel()$<br>$B.true = newLabel()$<br>$B.false = S.next$<br>$S_1.next = begin$<br>$S.code = label(begin) \parallel B.code$<br>$\parallel label(B.true) \parallel S_1.code$<br>$\parallel gen('goto' begin)$ |
| $B \rightarrow E_1 rel E_2$     | $B.code = E_1.code \parallel E_2.code$<br>$\parallel gen('if' E_1.addr rel op E_2.addr 'goto' B.true)$<br>$\parallel gen('goto' B.false)$                                                                              |
| $E \rightarrow E_1 + E_2$       | $E.addr = new Temp()$<br>$E.code = E_1.code \parallel E_2.code \parallel$<br>$gen(E.addr '=' E_1.addr '+' E_2.addr)$                                                                                                   |
| $  id$                          | $E.addr = top.get(id.lexeme)$<br>$E.code = ''$                                                                                                                                                                         |



### Three Address Code

```

L1:
if a > b goto L2 ] B
goto L3

L2:
t1 = d + e ] E5 ] E3 ] S1
t2 = c + t1
a = t2

goto L1

L3:

```